

Stack & Heap

Memory Management

Complete Reference Guide for Interviews & GATE

OS / Memory

DSA

Java / C++

GATE 2027

1. Introduction to Memory Management

When a program executes, the operating system allocates memory divided into distinct regions. The two most critical runtime regions are the **Stack** and the **Heap**. Understanding how they work is essential for writing efficient code, debugging memory errors, and acing technical interviews and competitive exams like GATE.

Key Insight: The stack is managed automatically by the compiler/runtime, while the heap requires manual management (C/C++) or garbage collection (Java/Python). Both reside in a process's virtual address space but serve very different purposes.

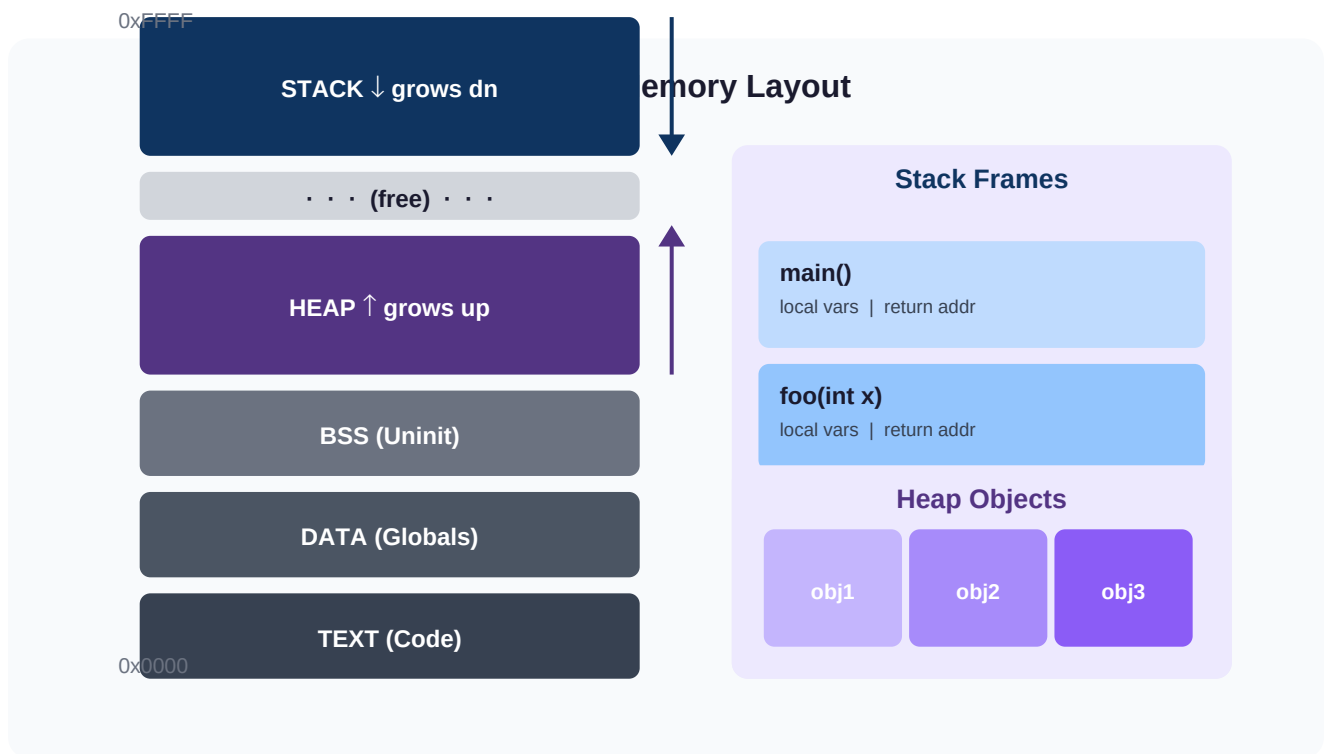


Figure 1 – Virtual Memory Layout of a Running Process

2. The Stack

2.1 What is the Stack?

The **Stack** is a contiguous block of memory that operates on a **Last-In, First-Out (LIFO)** principle. Every function call creates a new *stack frame* (activation record) that holds the function's local variables, parameters, and the return address. When the function returns, its frame is instantly reclaimed by decrementing the stack pointer.

Property	Detail
Location	High-address end of virtual memory; grows downward
Size	Fixed at process launch (typically 1 MB – 8 MB on Linux)
Allocation	Automatic – managed by compiler via SP (stack pointer)
Deallocation	Automatic – on function return
Speed	O(1) – extremely fast (single pointer move)
Thread Safety	Each thread has its own private stack
Lifetime	Limited to function scope
Overflow	Stack Overflow if limit exceeded

2.2 Stack Frame Structure

Each function call pushes a new **stack frame** onto the stack. The frame contains:

- **Return Address** – where to jump after the function completes
- **Saved Frame Pointer (SFP)** – previous frame's base pointer

- **Parameters** – arguments passed to the function
- **Local Variables** – variables declared inside the function
- **Saved Registers** – CPU register values to restore later

2.3 Stack in Java

In Java, the JVM manages a separate stack per thread. Primitive types and object *references* live on the stack; actual objects live on the heap.

```
public class StackDemo {
    public static int factorial(int n) { // new frame pushed
        if (n <= 1) return 1; // local var 'n' on stack
        return n * factorial(n - 1); // recursive frames stack up
    } // frame popped on return

    public static void main(String[] args) {
        int result = factorial(5); // 'result' stored on stack
        System.out.println(result); // prints 120
    }
}
```

2.4 Stack in C

```
void foo() {
    int x = 10; // x allocated on stack
    int arr[100]; // arr (400 bytes) on stack
    // x and arr freed automatically when foo() returns
}

// Stack Overflow example:
void infinite() {
    int big[1000000]; // huge array on stack
    infinite(); // recursive call → StackOverflowError
}
```

3. The Heap

3.1 What is the Heap?

The **Heap** is a large pool of memory used for **dynamic allocation** – memory whose size or lifetime is not known at compile time. Unlike the stack, heap memory must be explicitly requested and (in C/C++) explicitly released. In managed languages like Java, a **Garbage Collector (GC)** automatically reclaims unreachable objects.

Property	Detail
Location	Low-address region; grows upward toward stack
Size	Dynamic – limited by available virtual/physical memory
Allocation	Manual: malloc/new Managed: GC (Java, Python, C#)
Deallocation	Manual: free/delete Auto: GC or reference counting
Speed	Slower – allocator must find free block (O(n) worst case)
Thread Safety	Shared by all threads – requires synchronisation
Lifetime	Until explicitly freed or GC collects
Overflow	OutOfMemoryError / segfault if exhausted

3.2 Heap in C (Manual Management)

```
#include <stdlib.h>

int main() {
    // Allocate array of 10 ints on heap
    int *arr = (int*) malloc(10 * sizeof(int));

    if (arr == NULL) {
        // malloc returns NULL on failure
        return -1;
    }

    for (int i = 0; i < 10; i++) arr[i] = i * i;

    free(arr); // Must free - or memory leak!
    arr = NULL; // Good practice: avoid dangling pointer
    return 0;
}
```

3.3 Heap in C++ (RAII & Smart Pointers)

```

#include <memory>
#include <vector>

void modern_cpp() {
// unique_ptr - automatic delete when out of scope
auto p = std::make_unique<int>(42);
std::cout << *p << std::endl; // 42
// No delete needed! RAII handles it.

// shared_ptr - reference counted
auto sp1 = std::make_shared<std::vector<int>>>();
auto sp2 = sp1; // ref count = 2
// freed when both sp1 and sp2 go out of scope
}

```

3.4 Heap in Java (Garbage Collection)

In Java, **all objects** are allocated on the heap via `new`. The JVM's Garbage Collector (GC) automatically reclaims memory from objects with no live references. The heap is further divided into **Young Generation**, **Old (Tenured) Generation**, and (prior to Java 8) **PermGen / Metaspace**.

```

// Java - all objects live on the heap
public class HeapDemo {
public static void main(String[] args) {
// 'obj' reference on stack; MyObject on heap
MyObject obj = new MyObject("Hello");

// Array object on heap, reference on stack
int[] arr = new int[1000];

obj = null; // MyObject now eligible for GC
// GC will reclaim heap memory automatically
}
}

```

4. Stack vs Heap – Complete Comparison

Feature	STACK	HEAP
Structure	Contiguous; LIFO	Fragmented; dynamic
Size	Fixed (1–8 MB typical)	Dynamic (up to RAM limit)
Allocation Speed	O(1) – pointer move	O(n) – find free block
Deallocation	Automatic on return	Manual or GC
Thread Safety	Per-thread (safe)	Shared (needs locks)
Fragmentation	None	Yes (internal/external)
Typical Data	Local vars, params, frames	Objects, arrays, dynamic data
Access Pattern	Sequential (cache-friendly)	Random (cache unfriendly)
Overflow Handling	StackOverflowError/segfault	OutOfMemoryError
Java Specifics	Primitives + object refs	All objects + arrays
C++ Specifics	Automatic storage duration	Dynamic storage (new/delete)

5. Common Memory Errors

■ Stack Overflow

Occurs when stack size is exceeded. Caused by infinite/deep recursion or large local arrays.

```
void infinite() { infinite(); } // C++ StackOverflow
// Java: java.lang.StackOverflowError
// Fix: use iteration or increase stack size (-Xss in JVM)
```

■ Memory Leak

Heap memory allocated but never freed. Program's memory usage grows indefinitely.

```
int* p = new int[100];
// ... use p ...
// FORGOT: delete[] p; → memory leak
// Fix: use RAII / smart pointers / GC languages
```

■ Dangling Pointer

A pointer that refers to memory that has already been freed.

```
int* p = (int*) malloc(4);
free(p);
*p = 10; // Undefined behaviour - dangling pointer!
// Fix: set p = NULL after free
```

■ Heap Fragmentation

Free heap memory exists but in non-contiguous blocks; large allocations may fail.

```
// Caused by: many malloc/free of varying sizes
// Result: OOM even with total free memory available
// Fix: memory pools, slab allocators, or compact GC
```

■ Buffer Overflow

Writing past the end of a stack/heap buffer – leading to data corruption or security exploits.

```
char buf[8];
strcpy(buf, "Hello World!"); // 12 bytes → overflow!
// Fix: use strncpy / C++ std::string
```

6. Java Garbage Collection Deep Dive

Java's GC automatically manages heap memory. The heap is split into generations because *most objects die young* (the **Generational Hypothesis**).

Region	Size	GC Type	Promotion
Eden Space	Small	Minor GC	→ Survivor if alive
Survivor S0/S1	Small	Minor GC	→ Old Gen after N cycles
Old (Tenured) Gen	Large	Major/Full GC	–
Metaspace (JDK8+)	Dynamic	Full GC	Class metadata

Java GC Algorithms

GC Algorithm	JDK	Description	Best For
Serial GC	All	Single-threaded; stop-the-world	Small apps / JVM tuning
Parallel GC	JDK 8	Multi-thread; stop-the-world	Throughput-focused
G1 GC	JDK 9+	Region-based; low pause	Large heaps > 4 GB
ZGC	JDK 15+	Sub-millisecond pauses; concurrent	Ultra-low latency
Shenandoah	JDK 15+	Concurrent evacuation; low pause	Large heap, low latency

JVM Memory Flags

```
# Set initial and max heap size
java -Xms512m -Xmx2g MyApp

# Set stack size per thread
java -Xss2m MyApp

# Enable G1 GC with verbose logging
java -XX:+UseG1GC -Xlog:gc* MyApp

# Print GC statistics
java -verbose:gc -XX:+PrintGCDetails MyApp
```


7. GATE & Interview Quick Reference

GATE 2027 Focus Areas: Memory segmentation (OS), pointer arithmetic and dynamic allocation (C), Java heap/stack distinction, GC algorithms, memory leak detection, and complexity of allocator operations.

Q1. Where are local variables stored in Java?

On the JVM stack (in the current thread's stack frame). If the variable is an object reference, the reference is on the stack but the object itself is on the heap.

Q2. What is a memory leak in Java?

A situation where objects are still referenced (e.g., stored in a static collection) but never used again, preventing GC from collecting them. Common causes: static Maps, event listeners, thread-local variables not removed.

Q3. Difference between stack overflow and heap overflow?

Stack overflow: stack grows beyond its fixed limit (deep recursion). Heap overflow: heap memory exhausted (too many/large objects) → `OutOfMemoryError` in Java.

Q4. What is a dangling pointer?

A pointer/reference to memory that has been freed/deallocated. Accessing it causes undefined behaviour (C/C++) or is prevented by GC (Java).

Q5. What is RAII?

Resource Acquisition Is Initialization – a C++ idiom where resource lifetimes (heap memory, file handles) are tied to object scope. Destructors free resources automatically, preventing leaks.

Q6. How does G1 GC differ from Parallel GC?

G1 divides the heap into equal-sized regions rather than fixed Young/Old spaces. It prioritises regions with most garbage first, providing predictable pause times. Parallel GC does stop-the-world collections on fixed generations, optimising for throughput.

Q7. Can the stack ever be larger than the heap?

Theoretically yes, but in practice no. Stack is fixed and small (1–8 MB); heap can grow to available virtual memory (GBs). The OS enforces the stack limit at process creation.

Q8. What is internal vs external fragmentation?

Internal fragmentation: allocated block is larger than requested – wasted space inside block. External fragmentation: enough total free memory exists but not in a contiguous block. The heap suffers from both; the stack suffers from neither.

8. Memory Management Best Practices

C / C++

- ✓ Always pair every `malloc/new` with `free/delete`
- ✓ Set pointers to `NULL` after freeing to avoid dangling pointers

- ✓ Use RAII and smart pointers (`unique_ptr`, `shared_ptr`) in modern C++
- ✓ Prefer stack allocation for small, short-lived data
- ✓ Use Valgrind / AddressSanitizer to detect leaks and overflows
- ✓ Avoid Variable-Length Arrays (VLAs) on stack – use vectors

Java

- ✓ Avoid unnecessary object creation inside loops
- ✓ Remove objects from collections when no longer needed
- ✓ Use `WeakReference` / `SoftReference` for cache-like structures
- ✓ Close resources in finally blocks or use try-with-resources
- ✓ Profile with VisualVM / Eclipse MAT to detect heap leaks
- ✓ Tune GC flags: `-Xmx`, `-Xms`, `-XX:+UseG1GC` based on workload
- ✓ Prefer primitives (`int`) over wrappers (`Integer`) where possible

General

- ✓ Understand the cost of garbage collection pauses in latency-sensitive apps
- ✓ Minimise allocations in hot loops – object pooling helps
- ✓ Prefer iterative algorithms over recursive to reduce stack depth
- ✓ Use memory-mapped files for very large datasets instead of heap
- ✓ Monitor memory metrics in production (heap used/committed, GC time)

9. One-Page Cheat Sheet

Concept	Stack	Heap
Memory region	High addresses → grows ↓	Low addresses → grows ↑
Allocation	Compiler-managed (SP)	malloc/new / GC
Deallocation	Auto on function exit	free/delete / GC
Speed	O(1) – fastest	Slower – allocator overhead
Persistence	Function scope only	Until freed or GC
Java primitives	On stack	N/A (wrapped Integer on heap)
Java objects	Reference on stack	Object on heap
Thread model	1 stack per thread	Shared by all threads
GC target	No GC needed	GC targets heap
Error type	StackOverflowError	OutOfMemoryError
Fragmentation	None	Yes – internal & external
C keyword	auto (default)	malloc / calloc / new
C++ idiom	Local scope	RAII, smart pointers
Debug tool	GDB stack trace	Valgrind, VisualVM, MAT

Key Facts to Remember

- ◆ Stack grows **downward** (toward lower addresses); Heap grows **upward**
- ◆ Java: ALL objects (including arrays) live on the **heap**; primitives and refs on **stack**
- ◆ C malloc returns a **void***; always cast and always free
- ◆ GC algorithms: Serial → Parallel → CMS → **G1** (default JDK9+) → **ZGC** (JDK15+)
- ◆ Stack size tunable via **ulimit -s** (Linux) or **-Xss** (JVM)
- ◆ Heap size tunable via **-Xms** (initial) and **-Xmx** (maximum) JVM flags
- ◆ GATE tip: Memory segmentation order (low→high): Text → Data → BSS → Heap → Stack